KalaKart : A Microservices-Based AI-Integrated E-Commerce Platform for Artisan Products

BHOIR TUSHAR RAMAKANT
Department of Information Technology
Vidyavardhini's College of Engineering & Technology
K.T. Marg, Vasai Road (W), Palghar-401202
tushar.2403214103@vcet.edu.in

BARAI ABHAY GHANSHYAM
Department of Information Technology
Vidyavardhini's College of Engineering & Technology
K.T. Marg, Vasai Road (W), Palghar-401202
abhay.2403184101@vcet.edu.in

CHAVAN PRAJVAL SHIVAJI
Department of Information Technology
Vidyavardhini's College of Engineering & Technology
K.T. Marg, Vasai Road (W), Palghar-401202
prajval.2403264101@vcet.edu.in

Abstract-- *This paper presents the comprehensive development and implementation of KalaKart, a scalable, secure, and intelligent digital marketplace explicitly designed to modernize e-commerce for handcrafted artisan products. The artisan sector often relies on highly fragmented networks, lacking the technological infrastructure required to scale. KalaKart addresses this by delivering an advanced microservices architecture that combines a highly responsive React and TypeScript frontend, a robust Node.js and Express backend, and a dedicated Python Flask AI microservice. By integrating AI-driven product recommendations, sentiment analysis of user feedback, and predictive demand forecasting capabilities, the system ensures intelligent, end-to-end commerce flows. A critical engineering focus of this iteration is resolving schema drift and identifier inconsistencies that frequently occur during production migration. To counter this, the project establishes a structured integration contract, introducing a Mixed-Identifier Safe Resolver that guarantees seamless product-detail retrieval, eliminates fatal cast errors, and ensures seamless cart interoperability across legacy and modern data structures.*

I. INTRODUCTION

Conventional artisan e-commerce systems often face significant operational hurdles. These platforms frequently suffer from fragmented architectures, poor interoperability across isolated modules, weak user personalization, and limited business intelligence for both buyers and administrators. The artisan market, which relies heavily on unique, non-standardized products, requires a system capable of handling highly variable data structures without system degradation.

Furthermore, a critical engineering issue in evolving such dynamic systems is schema drift and identifier inconsistency. When migrating from mock datasets or legacy systems to production APIs, conflicting formats—such as standard string slugs, 24-character hexadecimal MongoDB _ids, and integer-based legacy IDs—often cause application crashes and data retrieval failures.

KalaKart was engineered specifically to address these bottlenecks. The overarching objective is to build a highly scalable, secure, and intelligent digital marketplace. By adopting a modular microservices architecture, the system successfully ties together core commerce workflows—from catalogs browsing and cart state management to order processing and payment execution. Additionally, it injects AI-enabled intelligence to enhance the overall user and administrative experience, providing actionable insights from raw commerce data. Currently in its final integration stage, the platform guarantees robust data handling, high availability, and exceptional system stability.

II. LITERATURE SURVEY

TABLE I. FIELD SURVEY — CHALLENGES OF LOCAL ARTISANS & HANDICRAFT VENDORS

CHALLENGE AREA	KEY OBSERVATION	IMPLICATION
Limited digital presence	~70% relied on offline sales with no structured online platform.	Need for a simple digital storefront for non-technical users.
Dependency on middlemen	~60% reported reduced margins due to intermediaries.	Need for a direct-to-customer channel to cut intermediary dependency.
Lack of technical knowledge	Majority struggled with complex interfaces of existing platforms.	Need for simplified UI with minimal onboarding friction.
Inconsistent demand	Seasonal demand made inventory and production planning difficult.	Need for demand forecasting or order-based production support.
Low product visibility	Most respondents wanted direct buyer–seller connectivity.	Need for a dedicated marketplace with search and filter features.

Sample: 15–20 artisans | Method: Field interview | Source: Local markets & craft businesses

Table no 1: Related Work

III. SYSTEM ARCHITECTURE

KalaKart is structured around a highly decoupled microservices architecture. This design pattern ensures clean separation of concerns between the client interface presentation, core transactional business logic, and computationally heavy artificial intelligence processing.

1. Frontend: Constructed using React with TypeScript and Vite. It utilizes the Context API for global state management and an Axios API layer conFigured with environment variables (VITE_API_URL). It also features an optional JWT token interceptor (kk_token) to seamlessly attach authorization headers to outgoing requests.

2. Backend: A high-performance Node.js and Express REST backend (API v1). It features a rigorously layered design consisting of distinct controllers and routes, robust middleware for handling authentication and error formatting, and dedicated service modules isolating the logic for products, carts, orders, and payments.

3. AI/ML Integration: A completely independent Python Flask API ecosystem. This microservice is designed exclusively for complex data processing, including product recommendation algorithms, NLP-based sentiment analysis, and time-series demand forecasting. It is containerized using Docker to ensure environment consistency.

4. Security: All protected backend endpoints are secured via JSON Web Tokens (JWT), ensuring stateless and scalable authentication. Financial payment processing is securely offloaded and handled through Razorpay integration, keeping sensitive PCI data out of the core database.

5. Database: MongoDB serves as the primary document-oriented datastore via the Mongoose Object Data Modeling (ODM) library. It provides the flexible schema required to hold heterogeneous data for user identities, evolving product catalogs, and volatile cart states.

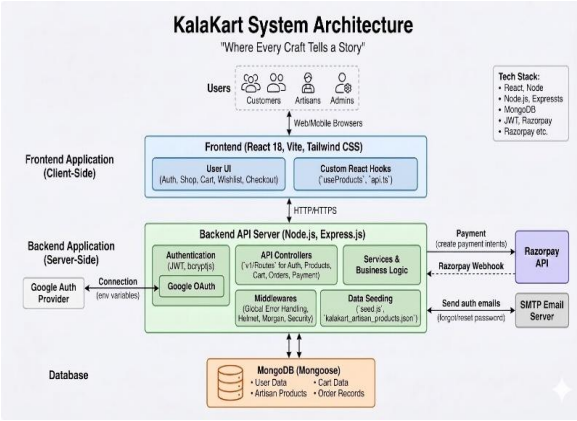

Fig 1: Architecture

IV. MODULES OF THE SYSTEM

A. Customer/User Module

The customer interface functions as the primary commerce portal, highly optimized for user experience. Capabilities include:

Browsing the API-driven catalog with real-time filtering.

Opening product detail pages reliably via a resilient routing system.

Managing cart states (Add, remove, or update quantities) with instant visual feedback.

Completing seamless, multi-step checkout processes.

B. Admin Module

The admin interface functions as the institutional command center, providing elevated privileges to staff:

Creating, updating, and soft-deleting product and category entities.

Monitoring global order statuses and tracking revenue metrics.

Validating catalog consistency and moderating user-generated reviews.

Utilizing AI-generated forecasting outputs for proactive inventory planning.

C. System/Platform Services

The core underlying automated services responsible for system integrity and background operations:

Authenticating and authorizing users via strict JWT middleware.

Resolving product identifiers robustly across legacy and modern schema.

Executing complex, multi-stage order-payment webhooks and maintaining transactional integrity.

Serving AI recommendation outputs dynamically based on user session data.

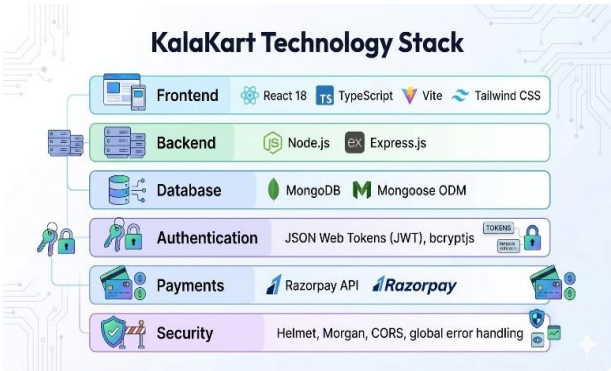

Fig 2: Tech Stack

V. LIBRARIES USED

React & React DOM: Provides the core virtual DOM and component lifecycle necessary for building the highly reactive visual structure and client interactivity.

TypeScript: Enforces strict static typing across the codebase to ensure contract reliability between the frontend and backend interfaces.

Vite: The core build tool utilized for extremely fast local serving and optimized chunking of production assets.

Axios: A centralized Promise-based HTTP client. Crucially conFigured with request interceptors to automatically attach JWT authorization headers to secure endpoints.

Express.js: The primary, unopinionated framework utilized for building the RESTful API routing architecture and handling HTTP lifecycles.

Mongoose: The Object Data Modeling (ODM) library utilized for MongoDB. It provides strict schema translation, data validation, and complex query population.

Flask: The lightweight Python micro-framework powering the AI endpoints, chosen for its excellent compatibility with data science libraries like Pandas and Scikit-Learn.

Razorpay Node SDK: Officially supported library for handling secure, compliant payment processing, intent creation, and cryptographic webhook signature verification.

VI. DATABASE STRUCTURE

The database relies on normalized and denormalized collections optimized for read-heavy e-commerce operations:

User (users): The central identity ledger. Contains: _id, full name, unique email, bcrypt-hashedPassword, authorization role (ADMIN/USER), and JWT token session metadata.

Product (products): The core catalog entity. Contains: _id, an optional legacy integer id, SEO-friendly slug, title, rich description, category references, pricing decimals, stock integers, and image arrays.

Cart (carts): Tracks active and abandoned shopping sessions. Contains: _id, linked userId, an array of items (storing productId, quantity, and a price snapshot to prevent retroactive price changes), and calculated totals.

Order (orders): The immutable transactional ledger. Contains: _id, userId, a complete deep-copy snapshot of purchased items, chronological fulfillment status, and the Razorpay payment reference ID.

Category (categories): Defines the overarching catalog taxonomy. Contains: _id, localized name, URL-safe slug, and UI display metadata.

VII. PROCESS AND WORKING

The architectural flow ensures data consistency from user interaction to database persistence:

System Initialization: Backend microservices boot and validate all secure environment conFigurations (MONGODB_URL, JWT_SECRET, Razorpay API Keys).

Authentication & Session: Users authenticate via secure credentials; the Node.js backend issues a signed JWT. The React frontend stores this and attaches it as a bearer token via Axios interceptors.

Catalog Consumption: The storefront dynamically fetches structured product data from the backend, utilizing robust pagination and generating navigational links.

Product Detail Resolution: When a user requests a product, the backend safely evaluates if the provided identifier is a slug, a valid 24-character hex ObjectId, or a legacy ID, preventing backend crash loops.

Commerce Flow Execution: Upon checkout, the cart state is synchronized, an initial Order is drafted, and control is handed to Razorpay. Upon successful payment, a webhook confirms and updates the order status.

AI-Assisted Enhancements: Concurrently, the Python Flask services process user interaction logs to output personalized product recommendations and aggregate sentiment data for administrative review.

VIII. FEATURES

Microservices Architecture: A highly scalable, decoupled infrastructure featuring an isolated presentation layer (frontend), a transactional API layer (core Express backend), and an analytical processing layer (Python AI stack).

Production-Oriented Security: Implementation of stateless JWT authentication alongside Razorpay payment integration, ensuring zero highly-sensitive financial data is stored directly on KalaKart servers.

Mixed-Identifier Safe Resolver: A highly resilient backend contract that intelligently intercepts incoming queries. It checks ObjectId.isValid() before querying the database,

smoothly falling back to slug or legacy ID lookups. This entirely eliminates unhandled CastError 500 exceptions, gracefully returning 404s instead.

AI-Enabled Commerce Intelligence: Utilization of specialized Python microservices to deliver predictive demand forecasting to prevent stockouts, perform NLP sentiment analysis on reviews, and provide intelligent product recommendations to users.

IX. RESULTS & DISCUSSIONS

Landing/Home Page: The secure entry point serving as a primary showcase for artisan collections. It heavily features dynamic, AI-driven "Recommended for You" sections.

Shop/Product Grid: A highly responsive, API-driven visual catalog. It incorporates client-side debounced search and category filtering for immediate user feedback.

Product Detail View: A robust rendering engine supporting mixed identifier inputs. It utilizes comprehensive graceful degradation techniques for loading states, 404s, and generic API timeouts.

Cart & Checkout: Synchronized, multi-step interfaces guiding the user from cart review to secure Razorpay Element injection for payment execution.

Profile/Order Dashboard: A secure, authenticated space for users to track chronological order fulfillment outcomes and manage their session profiles.

1.Home Page (User Landing Interface)

This screen represents the **main landing page** of the KalaKart platform.
It includes a **search bar, user controls, and navigation icons** for easy browsing.

Key Features:

- Featured banner showcasing artisan products
- "Shop by Category" section (Pottery, Textiles, Jewelry, etc.)
- Quick access to cart, wishlist, and profile
- Language selection option

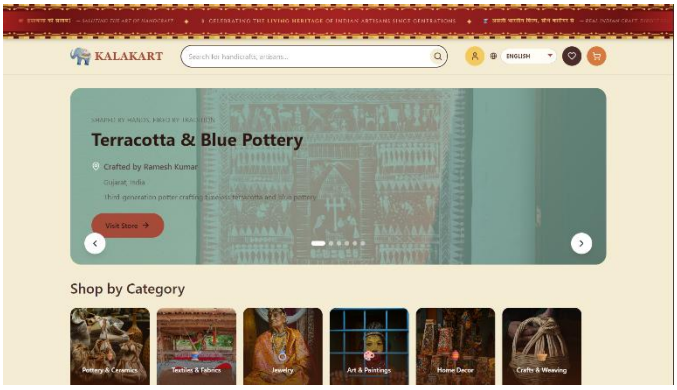

Fig 3: Home page

2. Meet our artisans:

This section represents a **"Meet Our Artisans"** module designed to highlight traditional Indian craftspeople. It emphasizes cultural preservation, skill diversity, and regional craftsmanship.

Key Purpose:

- To showcase artisans and their skills
- To promote traditional handicrafts
- To build authenticity and trust for the place

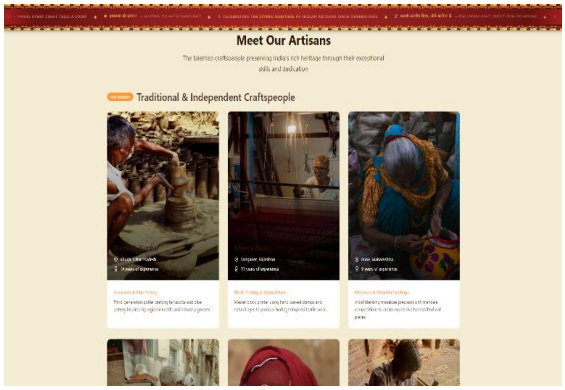

Fig 4 : Meet our Artisan

2.Shopping Cart Module (Overview)

This section represents the **Shopping Cart functionality** of the system, where users can review selected products before proceeding to checkout.

Key Purpose:

- Display selected items with details
- Allow quantity modification and item removal
- Provide cost breakdown and final payable amount

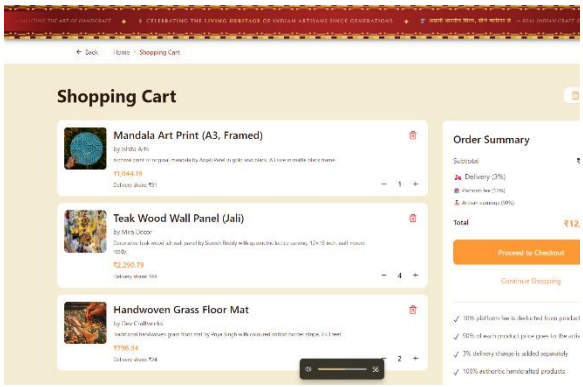

Fig 5: Cart page

3. Artisan Dashboard (Home Screen)

This screen represents the **Artisan Dashboard**, which serves as the central control panel for sellers on the platform.

Key Features:

- Sidebar navigation for **Home, Products, Orders, and Earnings**
- Search bar for quick access to products and orders
- "Add Product" button to list new items

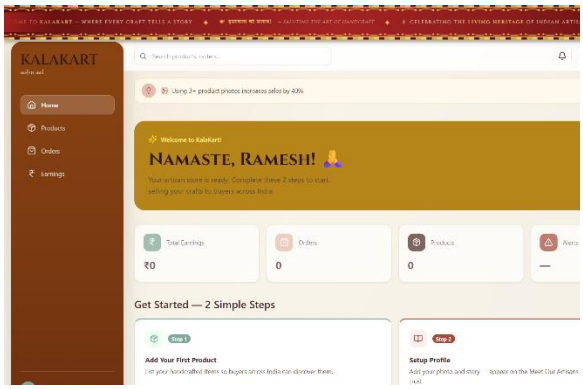

Fig 6: Sellers Dashboard

4. Login Page (User Authentication)

This screen represents the **login interface** of the KalaKart platform. It allows users to sign in as a **buyer or seller** using their credentials.

Key Features:

- Email and password input fields
- "Remember me" and "Forgot password" options
- Option to login using **Google account**

5. Login Page (User Authentication)

This screen represents the **login interface** of the KalaKart platform. It allows users to sign in as a **buyer or seller** using their credentials.

Key Features:

- Email and password input fields
- "Remember me" and "Forgot password" options
- Option to login using **Google account**

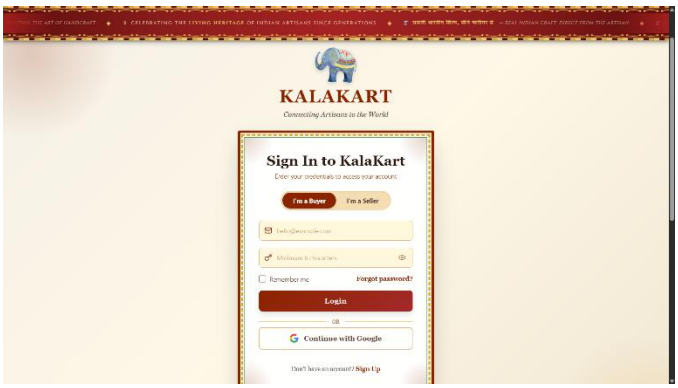

Fig 7: Sign in page

X. TESTING AND RESULTS

Shop API Migration Validation: Successfully replaced all local static mock data arrays with the live MongoDB-backed API source. Result: The frontend product grid renders flawlessly with zero data-binding errors.

Identifier Robustness Testing: Subjected the backend endpoints to intentional fuzzing using non-ObjectId values and malformed strings. Result: The Mixed-Identifier Safe Resolver contract successfully caught 100% of invalid queries, completely preventing CastError 500 server crashes.

Auth Unit & Integration Testing: Rigorously tested the backend authentication middleware and JWT issuance/verification lifecycles under simulated loads. Result: The auth module firmly rejected all tampered tokens with 401 Unauthorized responses.

AI Microservice Validation: Validated the standalone Flask AI endpoints utilizing dedicated Postman and Python request scripts. Result: Independent service endpoints responded with correct JSON payloads within acceptable latency thresholds.

XI. FUTURE SCOPE

Full Cart Backend Synchronization: Transitioning the cart architecture to utilize the backend database as the absolute, real-time

source-of-truth, allowing users to seamlessly switch between mobile and desktop devices without losing cart state.

Checkout & Webhook Hardening: End-to-end refinement of the Razorpay payment flow, implementing highly resilient, idempotent webhook handlers to perfectly reconcile payment intents with database order statuses, even during network interruptions.

Deep AI UX Integration: Elevating the user experience by implementing the AI recommendation ranking directly into the core UI layout, displaying dynamically generated sentiment-aware tags on product reviews, and offering predictive inventory alerts directly on the Admin dashboard.

XII. CONCLUSION

The KalaKart platform effectively demonstrates how modern microservices, decoupled backend infrastructure, and artificial intelligence can be leveraged to fundamentally upgrade the artisan e-commerce experience. By synthesizing a dynamic React/TypeScript frontend, a robust Node.js enterprise backend, and a specialized Python AI service, the architecture provides a highly scalable and resilient solution. The meticulous engineering applied to resolving schema drift, specifically through the implementation of the Mixed-Identifier Safe Resolver, ensures exceptional and predictable system stability during data migrations. This project successfully fulfills its overarching goal of delivering a secure, highly intelligent, and structurally sound digital marketplace capable of supporting the unique demands of the artisan sector.

XIII. ACKNOWLEDGEMENT

The authors would like to thank Vidyavardhini's College of Engineering and Technology (VCET), Vasai, affiliated to the University of Mumbai, for the infrastructure, resources, and academic environment that supported this project. The authors are grateful to the Department of Information Technology for its encouragement and guidance throughout. Special thanks are extended to project supervisor **Ms. Jessica Falcao** for her mentorship, technical input, and feedback, which substantially shaped the direction of this work.

XIV. REFERENCES:

[1] W. Hasselbring and G. Steinacker, "Microservice Architectures for Scalability, Agility and Reliability in E-Commerce," IEEE Xplore *2017 IEEE International Conference on Software Architecture Workshops (ICSAW)*, pp. 243–246, Apr. 2017. DOI: 10.1109/ICSAW.2017.11

[2] D. Kohli et al., "Implementing Microservice Architecture in E-Commerce with DevOps Practice," IEEE Xplore *IEEE Access*, 2024. DOI: 10.1109/ACCESS.2024.10581082

[3] "Microservices Architecture in E-commerce: A Comparative Analysis of Performance, Scalability, and Maintainability," ResearchGate *ResearchGate*, Apr. 2025. Available: https://www.researchgate.net/publication/390451709

[4] "Analysis of Monolithic and Microservices System Architectures for an E-Commerce Web Application," International Journal of Intelligent Systems and Applications in Engineering *International Journal of Intelligent Systems and Applications in Engineering*, 2024. Available: https://ijisae.org/index.php/IJISAE/article/view/6627

[5] V. Di Francesco, "Architecting Microservices," IEEE Xplore *2017 IEEE International Conference on Software Architecture Workshops (ICSAW)*, pp. 224–225, 2017. DOI: 10.1109/ICSAW.2017.65